



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 汇编程序 Hello World

专 业 班 级 通信工程 二班

学 生 学 号 202400120045

学 生 姓 名 刘哲旭

实 验 时 间 2025 年 3 月 日

实验报告

【实验目的】

- 1.掌握 window 的基本 Masm for Windows 集成实验环境 2015 的使用。
- 2.掌握在 xp 系统模拟 DOS 环境下 masm5 的编译和运行。
- 3.掌握 win11 使用 DOSBox 实现虚拟 DOS 环境下 masm5 的编译和运行。

【实验要求】

- 1.完成 DOS 环境和 Windows 环境的汇编程序的编译和执行并在屏幕上显示 HelloWorld。

【实验具体内容】

【第一个实验】

(1) 实验源代码（粘贴源代码）：

```
STACKS SEGMENT      ;定义栈段
    DW 256 DUP(?)    ;DW 即 define word（定义字），注意这里分配的空间为
                    256 个字，一个字为两个字节。DUP 全称 Duplicate（重复、复制），（?）代表保留空
                    间但未初始化的值，整句意思为批量分配 256 个未初始化的值的字的空间
```

```
    TOP EQU $-STACKS ;EQU 为 Equate,将右边的表达式赋值给左边的符号名
                    TOP,$为当前汇编位置计数器（Location Counter），代表“汇编器当前正在处理的字节
                    地址”。整句意思是计算“从段开始到现在一共用了多少字节”，也就是栈空间的总字节
                    数
```

```
STACKS ENDS          ;栈段结束
```

```
DATAS SEGMENT        ;定义数据段
```

```
    STRING DB 13,10,'Hello World!',13,10,$ ;定义字符串，内容为'Hello
                    World',在 ASCII 码中，13、10 分别代表回车和换行，$标志字符串的结束
```

```
DATAS ENDS           ;数据段结束的标签
```

```
CODES SEGMENT        ;定义代码段
```

```
    ASSUME CS:CODES,DS:DATAS,SS:STACKS ;ASSUME 是伪指令：告诉汇
                    编器在生成指令时，假设某些段寄存器指向哪些段。CS:CODES 表示代码段寄存器 CS
                    指向 CODES 段；DS:DATAS 表示数据段寄存器 DS 指向 DATAS 段（后面访问
                    STRING 变量时会用到）；SS:STACKS 则表示栈段寄存器 SS 指向 STACKS 段
```

START: ; 用户自定义，代表这里是程序入口

MOV AX,DATAS ;MOV 为 MOVE，表示将 DATAS 数据段的地址赋值给寄存器 AX

MOV DS,AX ;把刚才放进 AX 的段地址再传给寄存器 DS,从这一句之后，DS 指向了 DATAS 数据段

MOV AX,STACKS ;把栈段的段地址装入 AX

MOV SS,AX ; SS (Stack Segment) 指向我们定义的栈段

MOV SP,TOP ; SP (Stack Pointer) 指向栈的最高地址 (栈顶)

LEA DX,STRING ; LEA 表示 Load Effective Address (装载有效地址)，计算 STRING 的偏移地址 (相对于 DS)，放入寄存器 DX，STRING 之前指向定义过的字符串 Hello World!，所以执行后 DX 存放字符串 Hello World!的首地址。

MOV AH,9 ;将 9 赋值给寄存器 AX 高地址内存单元 AH

INT 21H ; INT 表示 Interrupt (软中断) ; 21H 是 DOS 的系统功能调用总中断号 ; CPU 跳转到中断向量表 21h 处执行 DOS 代码 ; DOS 根据上一步 AH 的值执行对应的功能，这次是显示字符串。

MOV AH,4CH ;将 4C 赋值给 AX 的高地址内存单元 AH

INT 21H ;再次调用 21h 中断，DOS 根据上一步 AH 的值执行对应的功能，这次 DOS 会结束当前程序

CODES ENDS ;结束代码段

END START ;标志程序整体结束

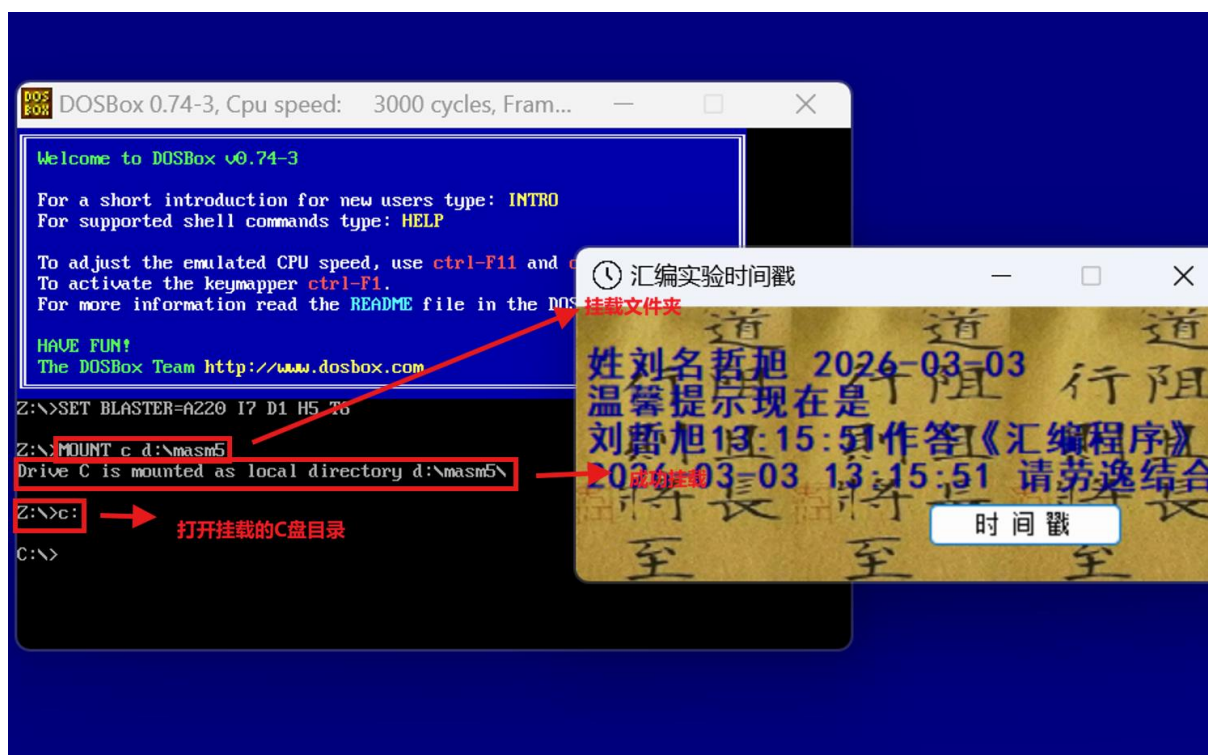
(3) 实验步骤

①在 DOSBox 的配置文件 DOSBox 0.74-3 Options.bat 末尾加入

mount c d:\masm5

c:

使 DOSBox 每次开始执行时自动挂载到 d 盘对应文件夹，并且直接定位 (cd) 到这个 c 盘里，不需要每次手动敲 mount 和 c: 命令。 ,如图所示



②将 HW.asm 放入 masm5 文件夹后，使用 dir 命令遍历 masm5 文件夹的目录，如图所示，红方框内为目标文件。



③输入 masm 命令后，填入 hw.asm 文件的名称，此后的 obj 和 lst 等文件默认跟随使用 hw，因此连续按三次回车跳过即可，如图所示。

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
.  OBJ      161 03-03-2026 23:24
LIB  EXE     32,150 31-07-1987 0:00
LINK  EXE     39,100 31-07-1987 0:00
MAKE  EXE     24,199 31-07-1987 0:00
MAKE  PIF      967 12-10-2002 11:27
MASM  EXE    65,557 31-07-1987 0:00
SETENU EXE    10,601 31-07-1987 0:00
15 File(s)      338,192 Bytes.
2 Dir(s)        262,111,744 Bytes free.

C:\>masm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Source filename [.ASM1: hw]
Object filename [hw.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:

51726 + 464818 Bytes symbol space free

0 Warning Errors
0 Severe Errors

C:\>S
    
```

输入 dir 遍历目录文件，此时发现有新生成的 obj 文件，如图所示

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
0 Warning Errors
0 Severe Errors

C:\>dir
Directory of C:\.
. <DIR>          04-03-2026  9:04
.. <DIR>         01-01-1980  0:00
CREF  EXE       15,830 31-07-1987 0:00
DEBUG EXE      20,634 08-02-2020 23:03
DEBUG32 EXE     90,720 01-08-1994 16:00
ERRROUT EXE      9,499 31-07-1987 0:00
EXEMOD EXE     12,149 31-07-1987 0:00
EXEPACK EXE     14,803 15-10-1987  5:00
HW    ASM        741 03-03-2026 23:19
HW    OBJ       161 04-03-2026  9:04
LIB   EXE      32,150 31-07-1987 0:00
LINK  EXE      39,100 31-07-1987 0:00
MAKE  EXE      24,199 31-07-1987 0:00
MAKE  PIF       967 12-10-2002 11:27
MASM  EXE     65,557 31-07-1987 0:00
SETENU EXE     10,601 31-07-1987 0:00
14 File(s)      337,111 Bytes.
2 Dir(s)        262,111,744 Bytes free.

C:\>S
    
```

④先调用 link 命令，输入 obj 文件名称 hw 对生成的 obj 文件进行进一步编译，再按三次回车跳过，即可生成如图所示的编译完成的 exe 文件

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
EXEMOD  EXE      12,149 31-07-1987 0:00
EXEPACK  EXE      14,803 15-10-1987 5:00
HW       ASM       741 03-03-2026 23:19
HW       OBJ       161 04-03-2026 9:04
LIB      EXE      32,150 31-07-1987 0:00
LINK     EXE      39,100 31-07-1987 0:00
MAKE     EXE      24,199 31-07-1987 0:00
MAKE     PIF       967 12-10-2002 11:27
MASM     EXE      65,557 31-07-1987 0:00
SETENV   EXE      10,601 31-07-1987 0:00
  14 File(s)      337,111 Bytes.
   2 Dir(s)       262,111,744 Bytes free.

C:\>link

Microsoft (R) Overlay Linker Version 3.60
Copyright (C) Microsoft Corp 1983-1987. All rights reserved.

Object Modules [OBJ]: hw
Run File [HW.EXE]:
List File [NUL.MAP]:
Libraries [LIB]:
LINK : warning L4021: no stack segment

C:\>S
    
```

输入 dir 遍历目录文件，此时发现有新生成的 obj 文件，如图所示

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
LINK : warning L4021: no stack segment

C:\>dir
Directory of C:\.
.          <DIR>      04-03-2026  9:05
..         <DIR>      01-01-1980  0:00
CREF      EXE       15,830 31-07-1987  0:00
DEBUG     EXE       20,634 08-02-2020 23:03
DEBUG32   EXE       90,720 01-08-1994 16:00
ERRROUT   EXE        9,499 31-07-1987  0:00
EXEMOD     EXE       12,149 31-07-1987  0:00
EXEPACK     EXE       14,803 15-10-1987  5:00
HW          ASM        741 03-03-2026 23:19
HW          EXE        1,001 04-03-2026  9:12
HW          OBJ        161 04-03-2026  9:12
LIB         EXE       32,150 31-07-1987  0:00
LINK        EXE       39,100 31-07-1987  0:00
MAKE        EXE       24,199 31-07-1987  0:00
MAKE        PIF        967 12-10-2002 11:27
MASM        EXE       65,557 31-07-1987  0:00
SETENV      EXE       10,601 31-07-1987  0:00
  15 File(s)      338,192 Bytes.
   2 Dir(s)       262,111,744 Bytes free.

C:\>S
    
```

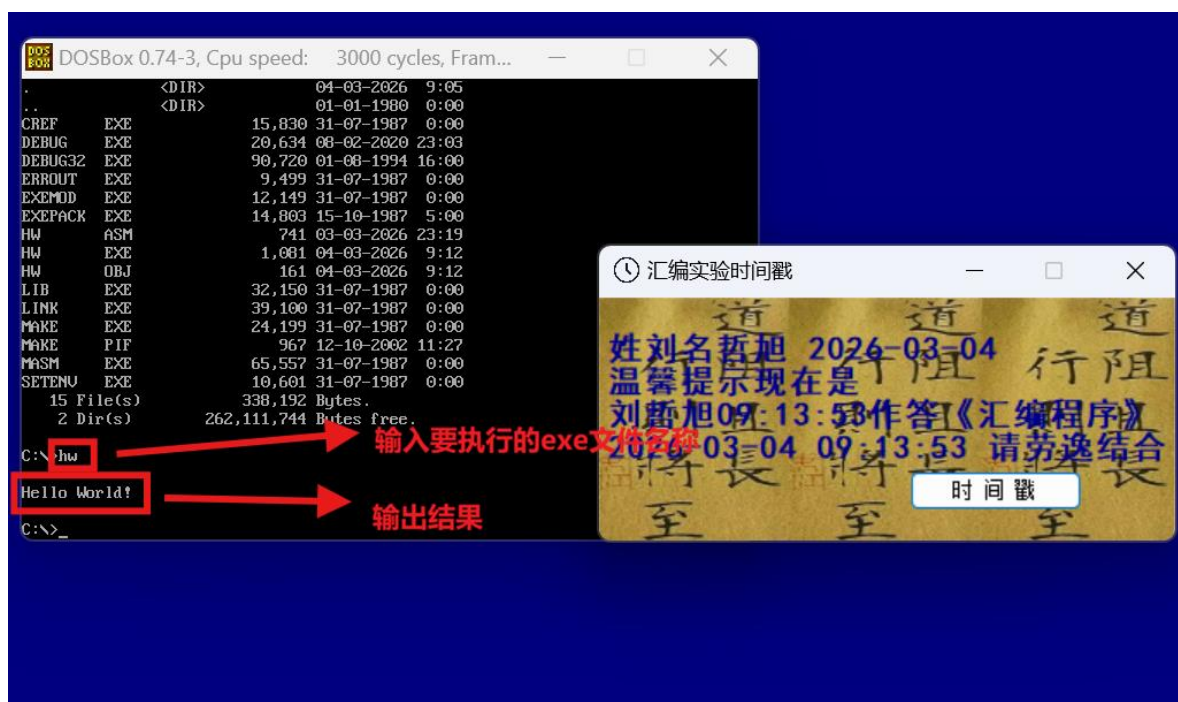
汇编实验时间戳

姓刘名哲旭 2026-03-04 09:13:09 作答《汇编程序》

温馨提示现在是 2026-03-04 09:13:09 请劳逸结合

时间戳

⑤最后输入 hw，系统运行 hw.exe，屏幕会输出 Hello World!，符合预期，试验成功，如图所示



【第二个实验】

(1) 实验源代码（粘贴源代码）：

例如：（第一次实验的源代码）

STACKS SEGMENT ;定义栈段

DW 256 DUP(?) ;DW 即 define word（定义字），注意这里分配的空间为 256 个字，一个字为两个字节。DUP 全称 Duplicate（重复、复制），(?) 代表保留空间但未初始化的值，整句意思为批量分配 256 个未初始化的值的字的空间

TOP EQU \$-STACKS ;EQU 为 Equate,将右边的表达式赋值给左边的符号名 TOP,\$为当前汇编位置计数器（Location Counter），代表“汇编器当前正在处理的字节地址”。整句意思是计算“从段开始到现在一共用了多少字节”，也就是栈空间的总字节数

STACKS ENDS ;栈段结束

DATAS SEGMENT ;定义数据段

STRING DB 13,10,'Hello World!',13,10,\$;定义字符串，内容为 'Hello World',在 ASCII 码中，13、10 分别代表回车和换行，\$标志字符串的结束

DATAS ENDS ;数据段结束的标签

CODES SEGMENT ;定义代码段

ASSUME CS:CODES,DS:DATAS,SS:STACKS ;ASSUME 是伪指令：告诉汇

编译器在生成指令时，假设某些段寄存器指向哪些段。CS:CODES 表示代码段寄存器 CS 指向 CODES 段；DS:DATAS 表示数据段寄存器 DS 指向 DATAS 段（后面访问 STRING 变量时会用到）；SS:STACKS 则表示栈段寄存器 SS 指向 STACKS 段

START: ; 用户自定义，代表这里是程序入口

MOV AX,DATAS ;MOV 为 MOVE，表示将 DATAS 数据段的地址赋值给寄存器 AX

MOV DS,AX ;把刚才放进 AX 的段地址再传给寄存器 DS,从这一句之后，DS 指向了 DATAS 数据段

MOV AX,STACKS ;把栈段的段地址装入 AX

MOV SS,AX ; SS (Stack Segment) 指向我们定义的栈段

MOV SP,TOP ; SP (Stack Pointer) 指向栈的最高地址（栈顶）

LEA DX,STRING ; LEA 表示 Load Effective Address（装载有效地址），计算 STRING 的偏移地址（相对于 DS），放入寄存器 DX，STRING 之前指向定义过的字符串 Hello World!，所以执行后 DX 存放字符串 Hello World!的首地址。

MOV AH,9 ;将 9 赋值给寄存器 AX 高地址内存单元 AH

INT 21H ;INT 表示 Interrupt（软中断）；21H 是 DOS 的系统功能调用总中断号；CPU 跳转到中断向量表 21h 处执行 DOS 代码；DOS 根据上一步 AH 的值执行对应的功能，这次是显示字符串。

MOV AH,4CH ;将 4C 赋值给 AX 的高地址内存单元 AH

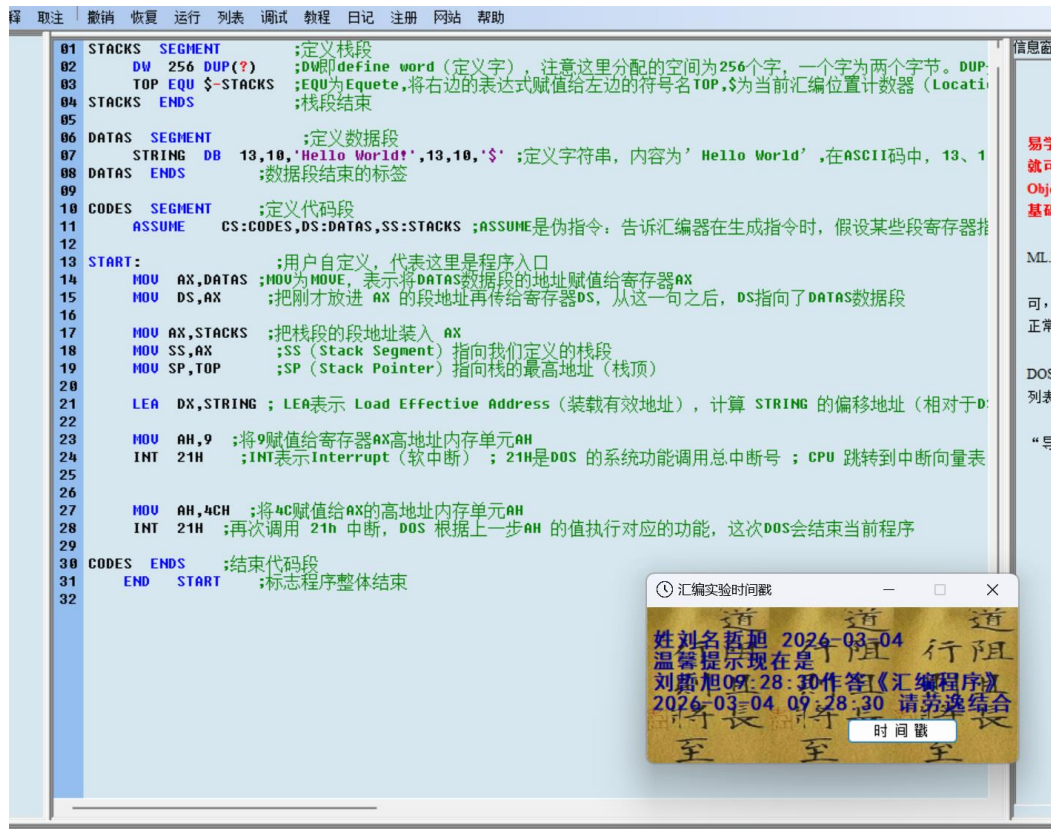
INT 21H ;再次调用 21h 中断，DOS 根据上一步 AH 的值执行对应的功能，这次 DOS 会结束当前程序

CODES ENDS ;结束代码段

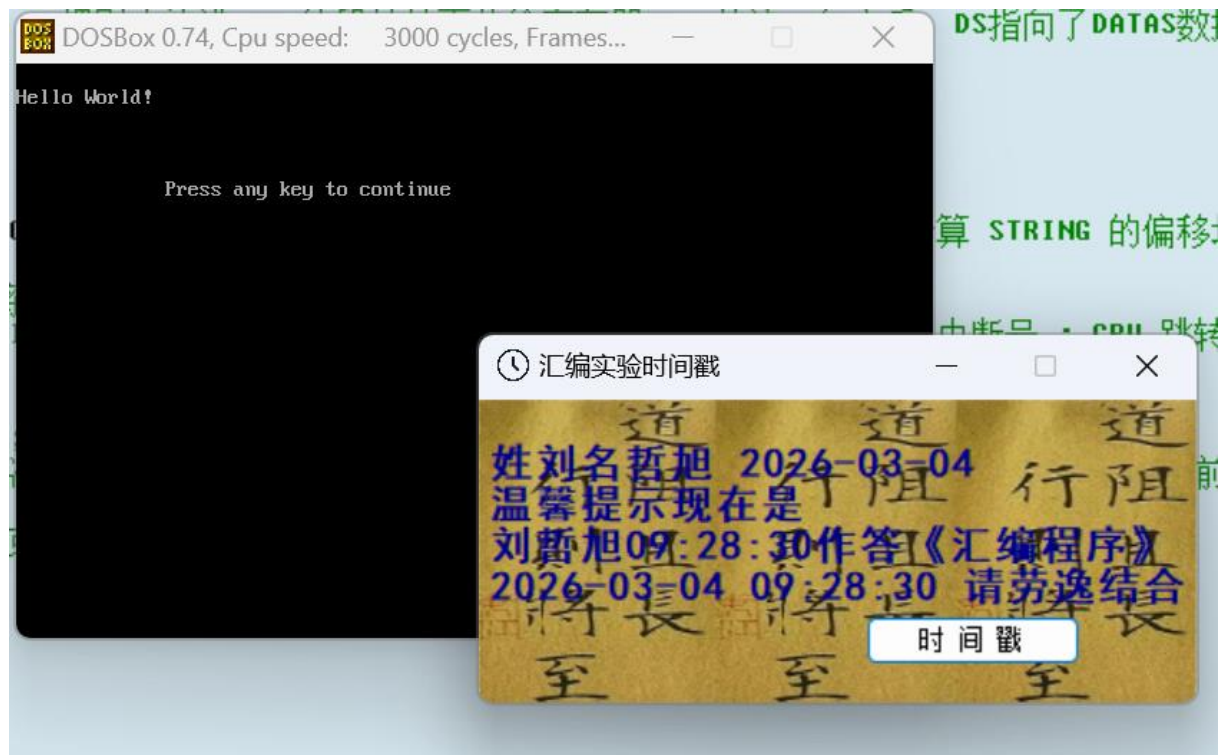
END START ;标志程序整体结束

（2）实验步骤

①打开 Masm fow Windows 集成实验环境 2015 后，将实验一中的 hw.asm 代码复制到代码区后并保存，如图所示



②点击运行键，弹出 Dosbox 窗口，输出如图所示的 Hello World!字样



【实验心得】

1. 我尝试下载安装 2023 版本的 Masm for Windows 进行编译，显示无法完成该操作，因为文件包含病毒和潜在的垃圾软件，经过查询资料后无果，遂安装使用 2015 年版本。

2. 文件夹的 hw.asm 代码和报告的实例代码存在差异，以实验报告模板的代码为例，如图所示：

例如：（第一次实验的源代码）

```

STACKS··SEGMENT··STACK··;栈段
·····DW··256··DUP(?)·····;注意 256 个字
STACKS··ENDS

```

之前没有STACK

```

DATAS··SEGMENT
·····STRING··DB··13,10,'Hello·World!',13,10,'$'
DATAS··ENDS

```

```

CODES··SEGMENT
·····ASSUME··CS:CODES,DS:DATAS
START:
·····MOV··AX,DATAS
·····MOV··DS,AX
·····LEA··DX,STRING
·····MOV··AH,9
·····INT··21H
···
·····MOV··AH,4CH
·····INT··21H
CODES··ENDS
·····END··START

```

缺少SS:STACKS

缺少: MOV AX,STACKS
MOV SS,AX
MOV SP,TOP

不同之处如下，一是上图代码的栈段定义中用了 STACK 属性，没有 TOP EQU \$-STACKS。二是 ASSUME CS:CODES,DS:DATAS（少了 SS:STACKS）；三是 MOV AX,STACKS MOV SS,AX MOV SP,TOP 这些内容同样缺少。经检验后再 DOS 和

Windows 环境下均能正常编译，输出预期的 Hello World!

借助 AI 工具，我了解到为什么不需要手动设置 SS 和 SP，却仍然能正常运行：

当把栈段定义为 **SEGMENT STACK**（注意加了 **STACK** 关键字）时：

MASM 会自动把这个段设置为程序的栈段；程序启动时，**系统会自动把 SS 指向这个栈段的段地址**；同时**自动把 SP 设置为栈段的最高地址**（也就是栈顶，从高地址向低地址生长）因此，不需要写 **MOV AX,STACKS / MOV SS,AX**、不需要写 **MOV SP,TOP** 甚至不需要 **TOP EQU \$-STACKS**（因为 MASM 自动知道栈顶位置）

只要程序中有**且仅有一个** **SEGMENT STACK** 的段，MASM 和 DOS loader 就会正确处理栈初始化

经过对比，图中的代码更短、更不容易出错、符合 MASM 的设计意图。

3.关于初次编译出现 1 warning error 的问题

在我第一次写完 hw.asm 的代码详细注释后，第一次编译时出现 **warning A4001: Extra characters on line**，经过自己多次检查未发现问题。后来上网查阅资料，询问 AI，发现自己在 **STACKS ENDS** 这一行的右面注释的时候输入的是中文分号；**TOP EQU \$-STACKS** 这一行的注释用的分号直接输入到指令行的末尾，经过修改之后，重新编译执行，正常无报错。由于假期预习不够充分，实验时操作不够熟练，由于编译成功并未发现 **warning error** 的问题，第二天重新编译时发现问题并更正。

4.在实验开始前，需要打开 DOS 进行挂载，对于挂载的概念我尚不清晰，经过查询资料得知：在 DOSBox 里，“挂载”（mount）是指把你电脑上真实的一个文件夹（或光驱、软驱镜像等）“映射”成 DOS 虚拟机里面的一个磁盘驱动器。

通俗来讲自己的电脑文件夹（这里是 D:\masm5）就像一个真实仓库。DOSBox 就像一台老式电脑，自己没有仓库。“挂载”相当于把真实仓库的门贴上一个标签（C:），然后告诉老电脑：打开 C 盘就能看见仓库里的东西

【课堂补充】

1.南桥与北桥

北桥（Northbridge）和南桥（Southbridge）是传统电脑主板芯片组（chipset）架构中的两个核心芯片组件，主要出现在 2000 年代到 2010 年代初的 Intel 和 AMD 平台上，用于实现 CPU 与其他硬件之间的数据通信和控制。也称 I/O Controller Hub 或 Fusion Controller Hub）是传统主板芯片组的核心逻辑架构组成部分。

北桥：位于主板靠近 CPU 的位置，主要负责高带宽、高性能组件之间的通信与控制，包括：

CPU 与主内存（DRAM）的直接接口（内存控制器）

显卡接口（如 AGP 或早期 PCI Express x16）

集成显卡（部分北桥集成 GPU 核心）

与南桥之间的内部高速通道（如 Intel 的 Hub Link 或 AMD 的 HyperTransport） 北桥直接连接前端总线（FSB）或类似高速总线，处理低延迟、高吞吐量的任务，是系统性能的关键瓶颈之一。

南桥：位于主板远离 CPU 的位置（通常偏下或右下角），负责中低速外围 I/O 设备的控制与管理，包括：

PCI/PCIe 低速通道

SATA/IDE 硬盘接口

USB 控制器

音频控制器、网卡、串口、键盘/鼠标控制器

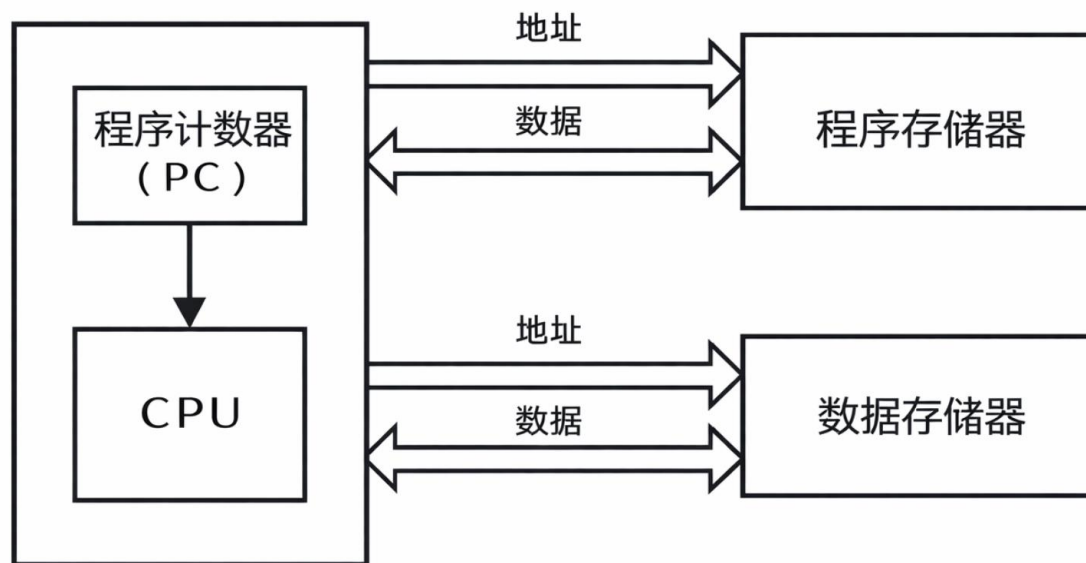
实时时钟（RTC）、电源管理（ACPI）、中断控制器等

南桥通过专用内部总线（如 Direct Media Interface）与北桥相连，不直接与 CPU 通信，所有数据需经北桥中转。

这种南北桥分离设计源于当时技术限制：将高性能部分（北桥）与稳定 I/O 部分（南桥）分开，便于优化布线、散热和成本。从 Intel Nehalem（2008 年起）和 AMD Athlon 64 时代开始，内存控制器等北桥核心功能逐步集成到 CPU 内部，北桥逐渐消失；Intel 从 5 系列芯片组起将南桥演变为 Platform Controller Hub（PCH），AMD 则称为 Fusion Controller Hub（FCH），现代主板芯片组多为单芯片（PCH）架构。

2.哈佛结构

哈佛结构是一种将程序指令存储和数据存储分开的存储器结构。哈佛结构是一种并行体系结构，它的主要特点是将程序和数据存储在不同的存储空间中，即程序存储器和数据存储器是两个独立的存储器，每个存储器独立编址、独立访问。与两个存储器相对应的是系统的四条总线，即程序的数据总线与地址总线，数据的数据总线与地址总线。



知识点

1. masm5 文件夹中包含后缀为 asm, obj, exe 的文件

(1) .asm 文件（源代码文件，源程序）

这是用文本编辑器编写的汇编语言源代码文件。内容包含汇编指令（如 MOV、INT）、伪指令（如 SEGMENT、ENDS、DB、DW、ASSUME、END）、宏定义、注释等。

作用：人类可读、可编辑的源文件，相当于 C 语言的 .c 或 .cpp

(2) .obj 文件（目标文件，对象模块，中间文件）

这是目标文件（也称对象文件或模块文件），由汇编器（如 MASM）对 .ASM 源文件进行汇编处理后生成。文件以二进制形式包含已转换的机器指令、符号表（变量名、段名、过程名等）、重定位信息以及可选的调试信息。它是汇编过程的中间结果，不能直接运行，必须通过链接器（如实验中所用的 LINK）与其他 .OBJ 文件或库文件进行链接，解决符号引用并生成最终的可执行程序。

(3) .EXE 文件（可执行文件、最终程序）

这是可执行文件，由链接器将一个或多个 .OBJ 文件（必要时结合库文件）链接而成，包含完整的程序代码、数据、栈信息、MZ 文件头（包含入口地址、重定位表等）。.EXE 文件是程序编译链接的最终产物，可以直接在 DOS 环境或 DOSBox 模拟器中运行，执行预设功能（如输出“Hello World!”字符串）。

2. 涉及到的核心汇编指令和伪指令作为知识点总结:

(1) 段定义相关伪指令

SEGMENT/ENDS 定义段的开始和结束, 用于划分栈段 (STACKS SEGMENT)、数据段 (DATAS SEGMENT) 和代码段 (CODES SEGMENT)。

作用: 将程序逻辑清晰分区, 便于段寄存器管理。

STACK 属性 (部分版本中使用, 如 SEGMENT STACK) 标记该段为栈段, MASM 会自动处理栈初始化 (SS 和 SP), 无需手动设置。

(2) 数据定义伪指令

DB (Define Byte) 定义字节数据, 常用于字符串, 如 DB 13,10,'Hello World!','13,10','\$'。其中 13 (回车 CR)、10 (换行 LF)、『\$』 (DOS 字符串结束符) 是标准用法。

DW (Define Word) 定义字 (2 字节) 数据, 常用于栈空间分配, 如 DW 256 DUP(?)。

DUP (Duplicate) 重复分配未初始化空间, 如 256 DUP(?) 表示重复 256 次未初始化的字 (常用于栈或缓冲区)。

EQU (Equate) 为表达式取别名, 如 TOP EQU \$-STACKS, 计算当前偏移量 (\$ 表示当前地址计数器), 得到栈顶相对偏移。

(3) 段寄存器假设伪指令

ASSUME 告诉汇编器段寄存器与段的对应关系, 例如: ASSUME CS:CODES, DS:DATAS, SS:STACKS 作用: 汇编器据此生成正确的段前缀, 避免手动每次加段覆盖。

(4) 数据传送与地址加载指令

MOV (Move) 数据传送指令, 用于:

段地址装入寄存器: MOV AX, DATAS、MOV AX, STACKS

段寄存器赋值: MOV DS, AX、MOV SS, AX

功能寄存器设置: MOV AH, 9、MOV AH, 4CH

LEA (Load Effective Address) 装载有效地址, 将变量的偏移地址装入寄存器, 如 LEA DX, STRING。常用于准备 INT 21H 的参数 (DX 指向字符串首地址)。

(5) DOS 系统功能调用 (中断)

INT 21H DOS 系统中断调用, 根据 AH 寄存器的值执行不同功能:

AH = 9: 显示以 '\$' 结束的字符串 (DX 指向字符串首地址)。

AH = 4CH: 正常终止程序并返回 DOS (退出码通常在 AL 中, 可省略)。

(6) 程序结束伪指令

END 标记源程序结束, 并指定程序入口点, 如 END START。作用: 告诉汇编器程序从哪个标号 (START) 开始执行。

3. DOS 系统功能调用 (中断 INT 21H)

INT 21H 是 DOS 系统提供的最主要中断服务（也叫 DOS 系统功能调用或 DOS API），几乎所有基本的 I/O、文件操作、日期时间、内存管理、程序控制等功能都通过它实现。调用方式统一为：把功能号装入 AH 寄存器（有时还需要其他寄存器参数），然后执行 INT 21H。

下面是 DOS INT 21H 的完整常见功能列表：

（1）程序终止与控制

AH = 00h: 程序终止（旧式，相当于 CP/M 兼容退出，返回 DOS）。

AH = 31h: 终止并驻留（Terminate and Stay Resident, TSR）。

AH = 4Ch: 正常终止程序并返回 DOS（带返回码在 AL，通常设 0 表示成功）——你的代码用的。

AH = 4Dh: 获取子程序返回码（AX 中）。

（2）字符/字符串 输入输出

AH = 01h: 从标准输入读字符 + 回显（AL 返回字符）。

AH = 02h: 向标准输出写单个字符（DL = 字符）。

AH = 05h: 向打印机输出单个字符（DL = 字符）。

AH = 06h: 直接控制台 I/O（DL = FFh 为读输入；否则写字符）。

AH = 07h: 直接从键盘读字符，无回显（不检查 Ctrl+Break）。

AH = 08h: 从标准输入读字符，无回显（检查 Ctrl+Break）。

AH = 09h: 输出以 '\$' 结束的字符串（DX 指向字符串首地址）——你的代码用的。

AH = 0Ah: 缓冲区输入字符串（支持多字符、编辑如 Backspace）。

AH = 0Bh: 检查标准输入是否有字符可用（AL = FFh 表示有）。

（3）文件/目录操作（DOS 经典功能，从 2.0+ 开始）

AH = 39h: 创建子目录（MKDIR, DS:DX 指向路径字符串）。

AH = 3Ah: 删除子目录（RMDIR）。

AH = 3Bh: 改变当前工作目录（CHDIR）。

AH = 3Ch: 创建/截断文件（CREAT, DS:DX 指向文件名, CX = 属性）。

AH = 3Dh: 打开文件（返回文件句柄在 AX）。

AH = 3Eh: 关闭文件（BX = 文件句柄）。

AH = 3Fh: 从文件/设备读数据（BX=句柄, CX=字节数, DS:DX=缓冲区）。

AH = 40h: 向文件/设备写数据。

AH = 41h: 删除文件（DS:DX 指向文件名）。

AH = 42h: 文件定位（Seek, 移动文件指针）。

AH = 43h: 获取/设置文件属性。

AH = 45h: 复制文件句柄。

AH = 46h: 重定向句柄。

AH = 47h: 获取当前目录路径。

AH = 56h: 重命名文件/目录。

AH = 57h: 获取/设置文件日期/时间。

(4) 日期/时间操作

AH = 2Ah: 获取系统日期 (CX=年, DH=月, DL=日, AL=星期)。

AH = 2Bh: 设置系统日期。

AH = 2Ch: 获取系统时间 (CH=时, CL=分, DH=秒, DL=1/100 秒)。

AH = 2Dh: 设置系统时间。

(5) 系统/磁盘信息

AH = 0Dh: 磁盘复位 (刷新缓冲)。

AH = 0Eh: 选择默认驱动器 (DL = 驱动器号, 返回上一个驱动器数)。

AH = 19h: 获取当前默认驱动器 (AL 返回)。

AH = 25h: 设置中断向量。

AH = 30h: 获取 DOS 版本号 (AL=主版本, AH=次版本)。

AH = 35h: 获取中断向量。

AH = 36h: 获取磁盘剩余空间 (DL=驱动器, 返回簇数等)。

AH = 4Eh: 查找第一个匹配文件 (FindFirst)。

AH = 4Fh: 查找下一个匹配文件 (FindNext)。

(6) 其他常用或高级功能

AH = 0Ch: 清空标准输入缓冲区并执行指定功能 (常与 01h/07h/08h/0Ah 结合)。

AH = 48h: 分配内存块。

AH = 49h: 释放内存块。

AH = 4Ah: 修改已分配内存块大小。

AH = 59h: 获取扩展错误信息 (DOS 3.0+)。

AH = 62h: 获取当前 PSP (Program Segment Prefix) 地址。